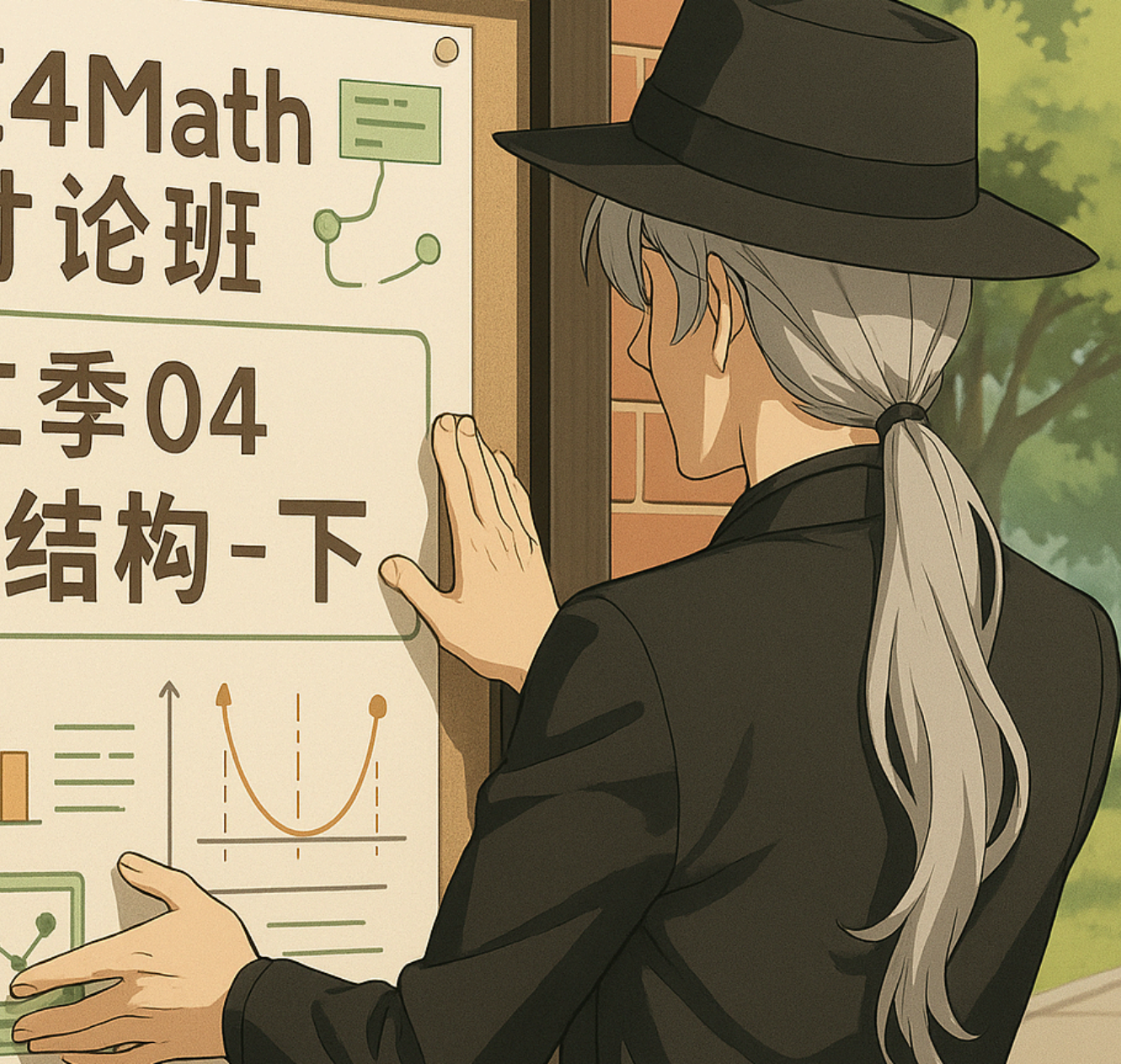
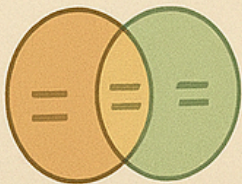
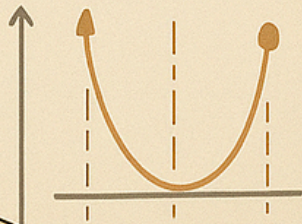
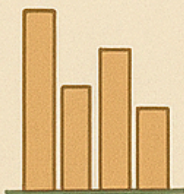
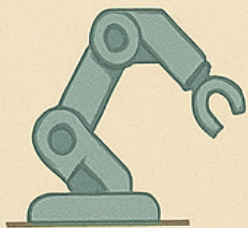


AI4Math 讨论班



第二季04 模型结构-下



模型结构-下

AI4Math 讨论班第二季-04

晚星

求真书院

2025-11-01

前言



请帮助我执行一个图像分类的任务，告诉我图像当中的物体是什么，并且为我详细介绍一下它的相关情况吧

图像中主要的物体是一只狗。

🐶 图像内容分析

这是一只中等体型、毛发呈棕色的狗，正站在积雪的院子里。狗的嘴边和鼻头上沾着一些雪，看起来刚刚在雪地里玩耍过。背景中可以看到被厚厚积雪覆盖的桌椅和植物，说明这是冬季拍摄的户外场景。

🐶 狗的特征与可能的品种

从外观来看，这只狗的体型修长、毛发中等长度、胡须明显、耳朵微微下垂，整体特征类似于爱尔兰梗（Irish Terrier）或混种梗犬（Terrier mix）。不过，由于没有确切的血统信息，也可能是混血犬。

爱尔兰梗以其：

- 勇敢活泼的性格；
- 强烈的好奇心；
- 对人友善、忠诚的天性；
- 以及适合寒冷气候的密集粗毛皮层而闻名。

这些特征都与图中狗狗的外貌和环境相吻合。

🌐 与环境的互动

狗站立在雪地上，显得既兴奋又好奇。积雪覆盖的地面和家具表明温度较低，但它似乎并不畏寒。这说明它可能有良好的防寒毛发层，属于适应寒冷环境的犬种。



可以帮助我查找一下这张绘画作品的出处吗？



这张插画上有一个明显的签名：@_yuk0ring

这是一位活跃于 Pixiv / Twitter（现 X）的日本插画师，经常发布这种水彩感、柔和色调的和风少女绘画。

✅ 你看到的这张图极有可能来自 Pixiv 用户「ゆこりん (yuk0ring)」的作品集。

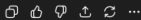
她的 Pixiv ID 常见为类似 101137977 等作品编号（图中风格与她的作品非常一致）。

✅ 如果你想确认具体出处，我可以帮你：

我可以进一步为你查找：

- Pixiv 作品页面链接
- 是否发表过在 Twitter (X)
- 作品标题 / 主题（例如中秋、和服、美少女等）
- 是否为画集或活动插画

👉 需要我继续为你查到具体链接或作品编号吗？



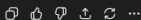
帮助我查找一下Pixiv作品的页面链接吧

Thought for 24s >

找到了！这张图对应的 Pixiv 作品页是：

<https://www.pixiv.net/artworks/101137977>

作品标题为「十五夜 * 透明水彩」，作者是 倭子鈴 (@_yukoring)。以上信息可在多处转载页核对到同一作品 ID 与标题、作者标注。



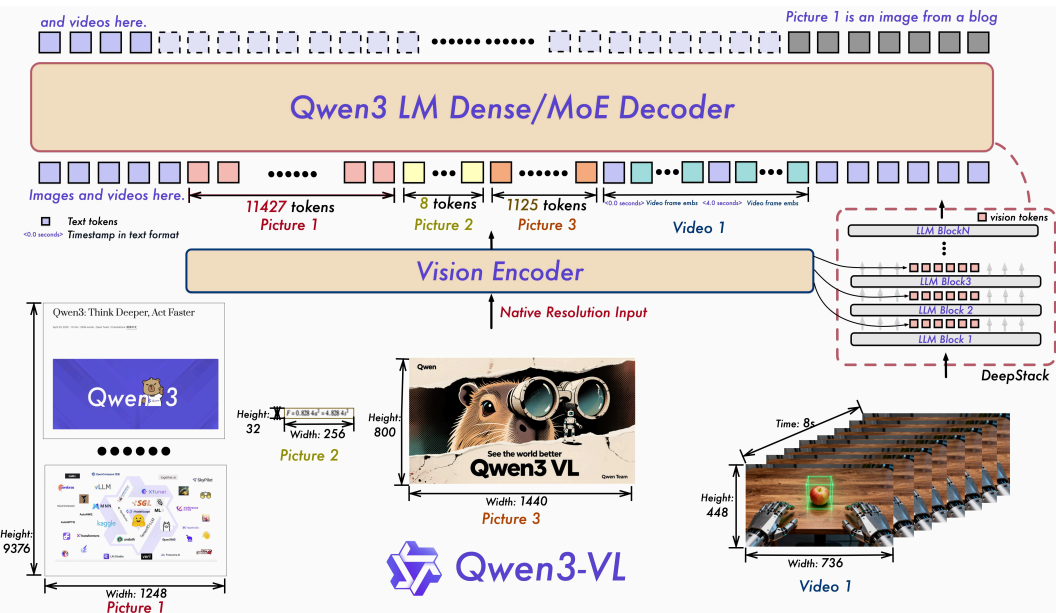
Qwen3-VL

```
{ "characters": [ { "name_en": "Giyu Tomioka", "name_zh": "富冈义勇", "bbox_2d": [0, 54, 473, 999] }, { "name_en": "Sanemi Shinazugawa", "name_zh": "不死川实弥", "bbox_2d": [218, 53, 478, 567] }, { "name_en": "Mitsuri Kanroji", "name_zh": "甘露寺蜜璃", "bbox_2d": [430, 83, 565, 414] }, { "name_en": "Gyomei Himejima", "name_zh": "悲鸣屿行冥", "bbox_2d": [544, 0, 671, 357] }, { "name_en": "Obanai Iguro", "name_zh": "伊黑小芭内", "bbox_2d": [645, 115, 793, 653] }, { "name_en": "Kyojuro Rengoku", "name_zh": "炼狱杏寿郎", "bbox_2d": [764, 36, 999, 579] }, { "name_en": "Muichiro Tokito", "name_zh": "时透无一郎", "bbox_2d": [197, 348, 473, 999] }, { "name_en": "Shinobu Kocho", "name_zh": "蝴蝶忍", "bbox_2d": [433, 324, 683, 999] }, { "name_en": "Tengen Uzui", "name_zh": "宇髄天元", "bbox_2d": [586, 445, 999, 999] } ] }
```



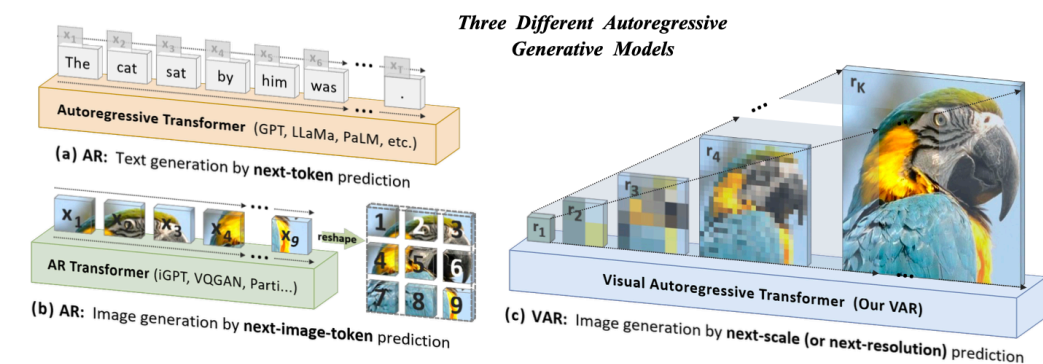
如此这一切能力的实现都依赖于如今的多模态大语言模型技术 (Multimodal LLM, MLLM)，而它背后的模型结构无一例外都是各种 **Transformer** 或其变体。

不过在如今，绝大多数的 MLLM 都仅支持图像等多模态输入的能力，而在多模态输出层面上则始终存在一定的困难。输入层面的通常做法是额外训练一个 Encoder 将各种模态的输入统一映射到语义空间。



2024 年的 CVPR best paper, VAR (Vision AutoRegressive model) 这篇论文则首次使用 Transformer 架构加上类 GPT 的自回归生成模式实现了图像生成任务, 并且取得了远超同期 diffusion model 的效率和性能表现。

这为 CV 和 NLP 的最终统一提供了一条可能路线, 但真正的未来还有待探索。



于 2025 年 3 月 25 日发布的 GPT image generation 功能则将视觉生成也整合进入了 ChatGPT 当中, 并且实现了图像生成质量与可控性的显著提升。

在此之后我们也见到了 Bagel, gemini nano banana 等类似竞品诞生。

而我们今天的主题就是
Transformer 模型架构

语言模型

与我们此前所介绍的图像识别等模型相比，处理语言任务时会存在两个关键的不同点

- 语言符号离散，无法直接由神经网络处理
- 语言文本是变长的序列

在这其中，语言的离散性目前主要依靠 tokenizer 以及 token embedding 处理。而在这之后则通常会采用循环神经网络 (Recurrent Neural Network, RNN) 或者 Transformer 这样针对设计的神经网络进行处理。

广义上针对序列建模的模型都可以称为语言模型。

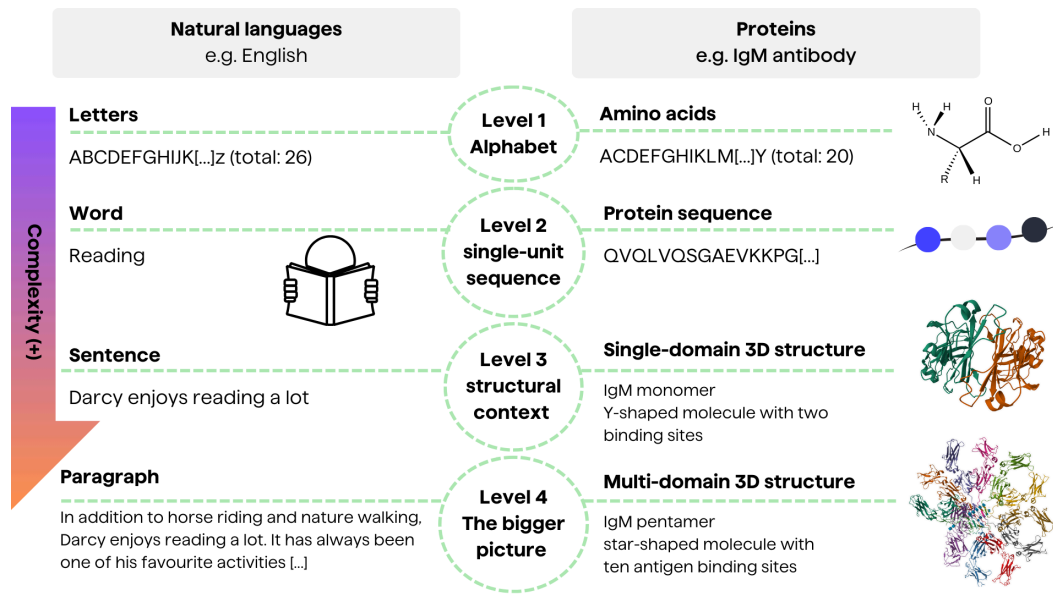
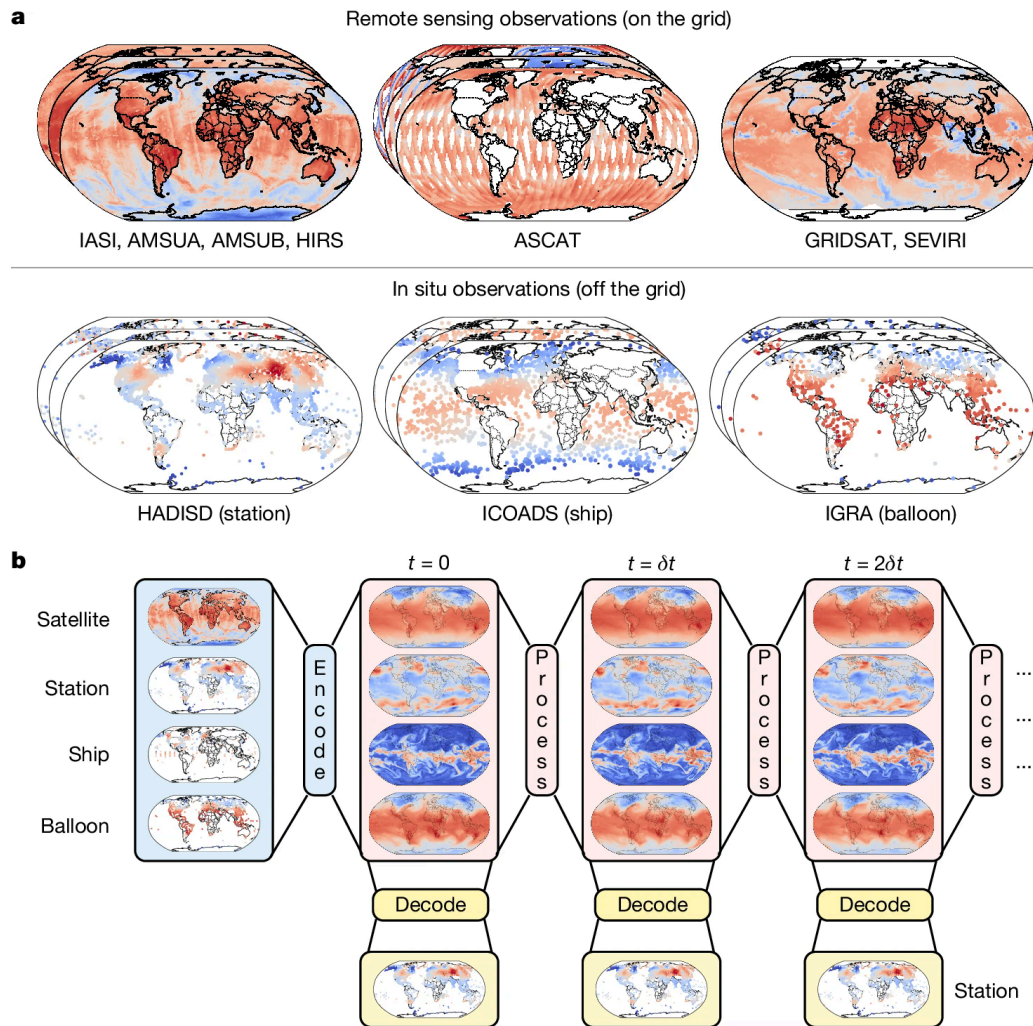


Figure 1: AI4Science 当中就常常能看到 Protein Language Model 这一概念

针对序列数据的一般处理方式就是将其拆分成数个单独、大小相同的基本单元进行处理。如此一来我们就只需要处理定长的数据即可。

而作为序列数据而言，另一项最关键的目标则是要处理各个单元之间的相关性。对于文本而言，一个基本处理单元就是一个 token。

当前的 AI 天气预报通常同样会采用自回归方式进行，如右图所示。



当前 GPT 类的大模型最常使用的 Tokenizer 方案是 Byte Pair Encoding (BPE) 算法，它直接基于统计给出可能的 token 词表。

- BPE 首先会初始化所有可能的 Byte 作为初始的 token 列表
- 统计给定的文本数据集当中出现频率最高的 token 组合
- 合并频率最高的 token 构造一个新的 token 添加到列表当中
- 直到词表 token 数量达到预设值

这样的 token 切分算法并不保证语义合理性，但是能够较好地确保工作效率。

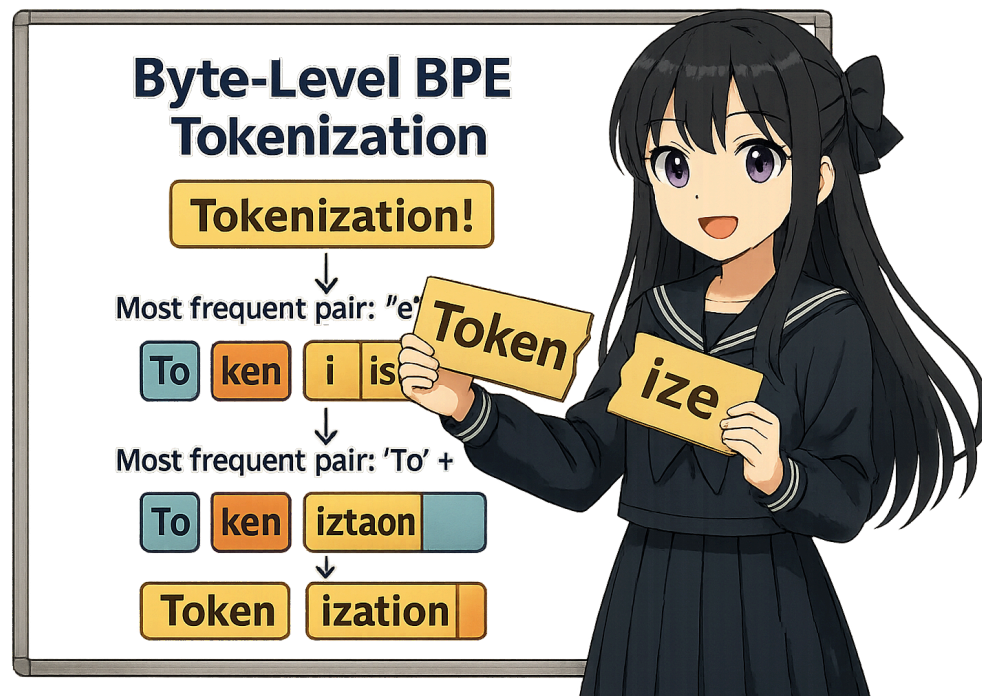


Figure 2: BPE 算法的演示

循环神经网络

循环神经网络最早于 2011 年左右提出，它的大致实现方式就是通过线性层逐一处理每个 token，并将中间信息以一个数值张量（也即隐藏层 H_t ）的形式传递到下一次计算当中。

在右侧公式当中， X_t 表示对应位置输出的词向量， H_t 就是隐藏层数据， W_{xh} , W_{hh} , W_{hq} 分别是三个参数矩阵， b_h , b_q 是两个偏置项。

我们通过训练使得输出 O_t 近似拟合 $t + 1$ 时刻的 token 出现的概率，由此即可实现自回归生成。（当然实际上也存在很多其他用法）

RNN 的两个主要计算公式：

$$H_t = \varphi(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

$$O_t = H_t W_{hq} + b_q$$

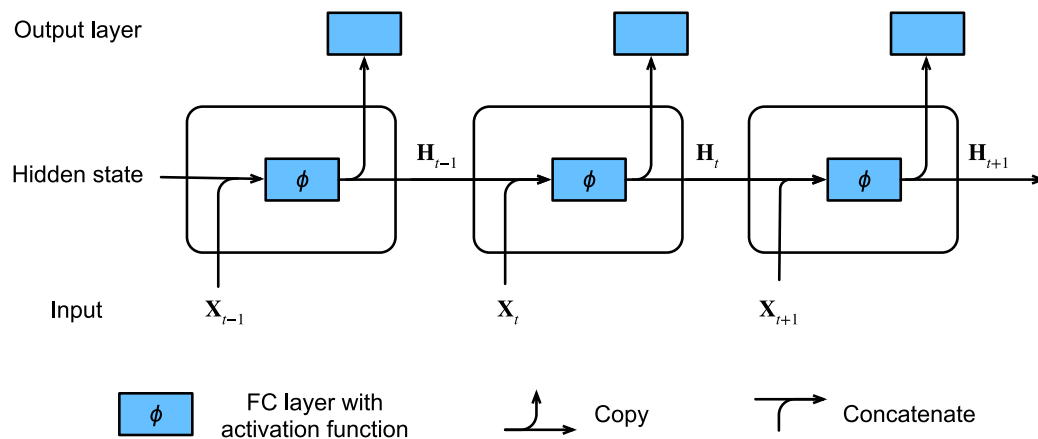


Figure 3: 经典 RNN 计算流程示意图

我们在示例代码“rnn_demo.py”当中就实现了一个最简单的 RNN 模型，它使用的是直接的单层 RNN 作为语言模型，并且使用了最简单的字符级 tokenizer (并没有使用 BPE 算法，而是直接将单个字符作为 token)。

其模型结构在 PyTorch 当中的主要实现方式如右所示，这里的“input_size”所指的就是词表大小。我们随后就在莎士比亚全集上就训练这个模型来实现文本生成任务。

```
# class CharRNN(nn.Module):
def __init__(self, input_size,
hidden_size, output_size):
    super(CharRNN, self).__init__()
    self.hidden_size = hidden_size
    self.embedding =
nn.Embedding(input_size, hidden_size)
    self.rnn = nn.RNN(hidden_size,
hidden_size, batch_first=True)
    self.fc = nn.Linear(hidden_size,
output_size)

def forward(self, x, hidden):
    x = self.embedding(x)
    out, hidden = self.rnn(x, hidden)
    out = self.fc(out)
    return out, hidden
```


此处我们每次生成时均使用相同的前缀：

“ROMEO:”

并要求我们的语言模型在它之后补全生成 100 个 token (字符)。可以看到随着训练的进行，生成文本的质量有了显著提高 (虽然还是在胡说八道 x)。

在未经训练时 RNN 的输出基本为纯乱码，训练一轮后似乎开始有了英文的感觉，到 20 轮后 RNN 就已经可以学会很多词汇的正确拼写了。

Epoch 0:

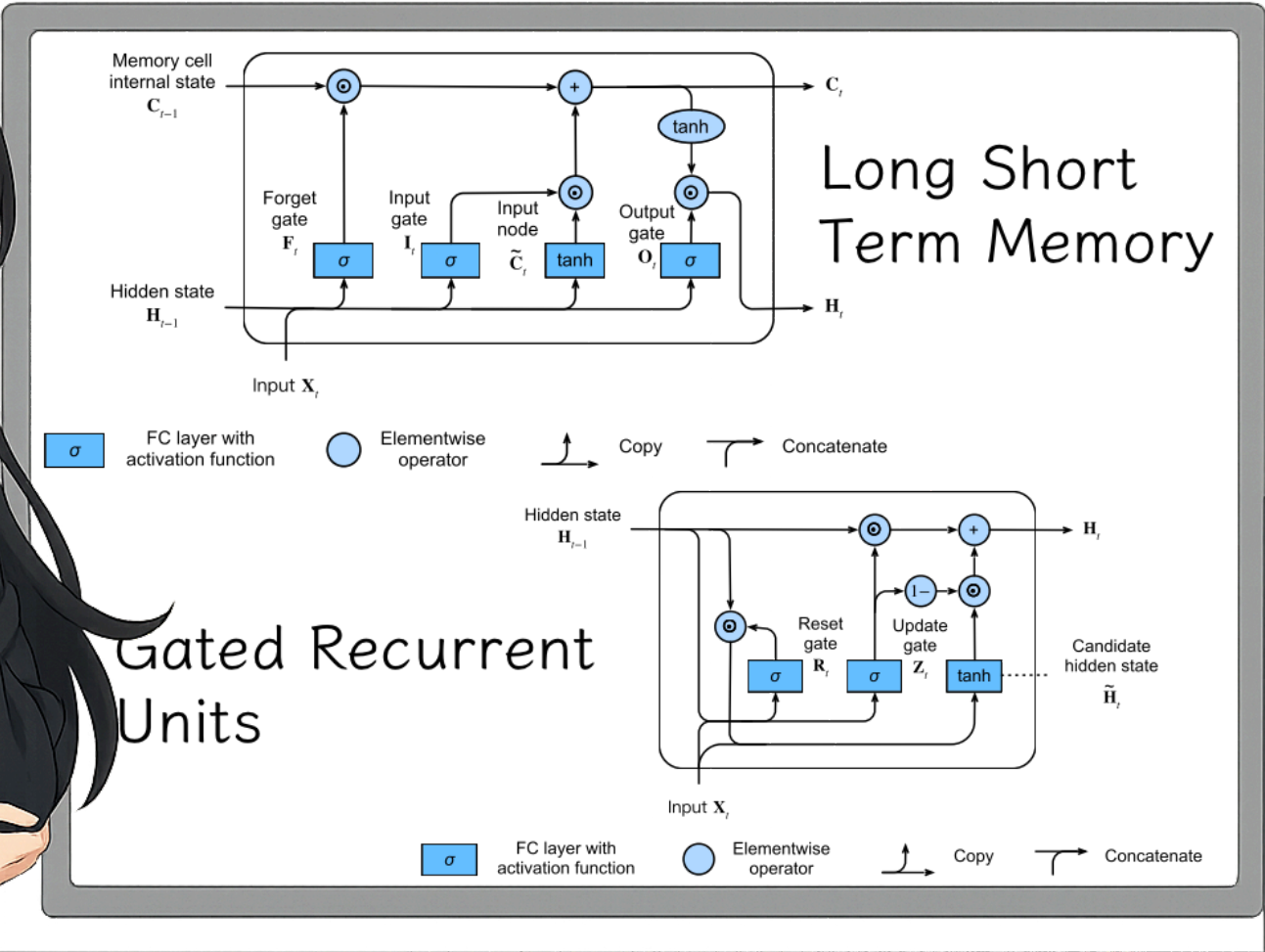
e lsz.raJEaQrhRFW;H-ni!
Vahgytpnha,KKUhtGh3kjb3 f.?LcU'3dqCthF
fbym. ?jjgs.YNrr. Z?&!U.-HtK-MURLZBT

Epoch 1:

Sill, we heart, of but to excold is him. Lided
and for Mare do be shang againg Sor the
prist, and to

Epoch 20:

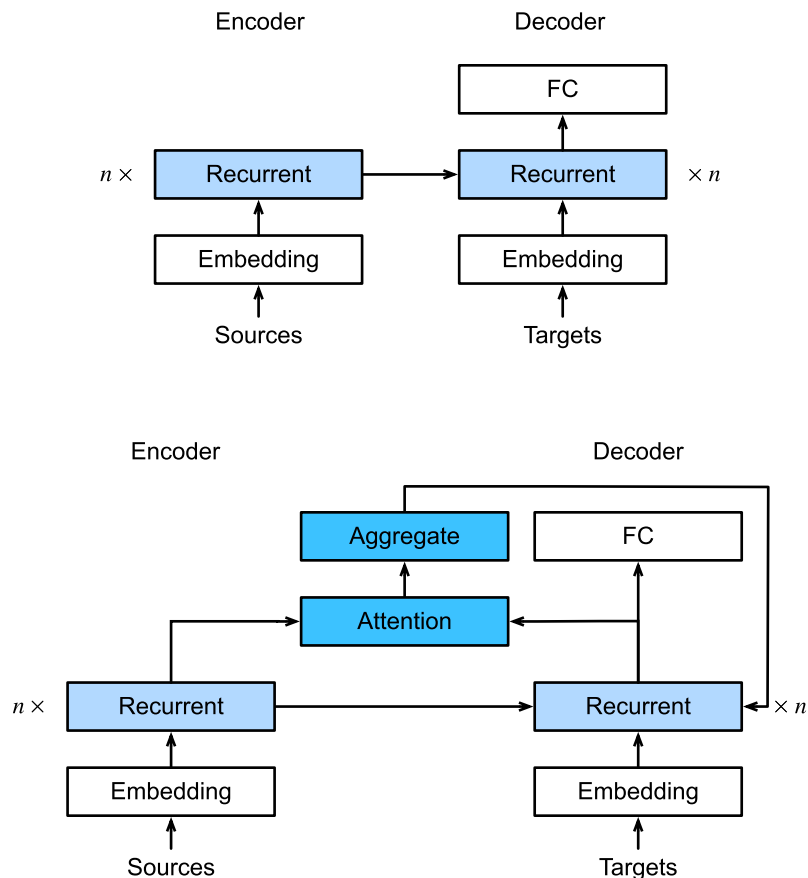
If he twenty more man, I crown: Say no
hation, therefore to the good some mayst
that tears the death



Attention 机制最早提出于 2014 年，它来源于早期 NLP 的核心领域 - 机器翻译。

经典的 RNN 机器翻译模型使用一种 seq2seq 的工作模式，将输入的文本序列经过 RNN 编码与解码后翻译为另一种语言的文本序列。

而在直觉上，当我们在执行翻译任务时，实际上我们总需要选择性关注一小部分相关的 token，但 RNN 却始终平等地处理每一个 token。这或许有很大的优化空间。



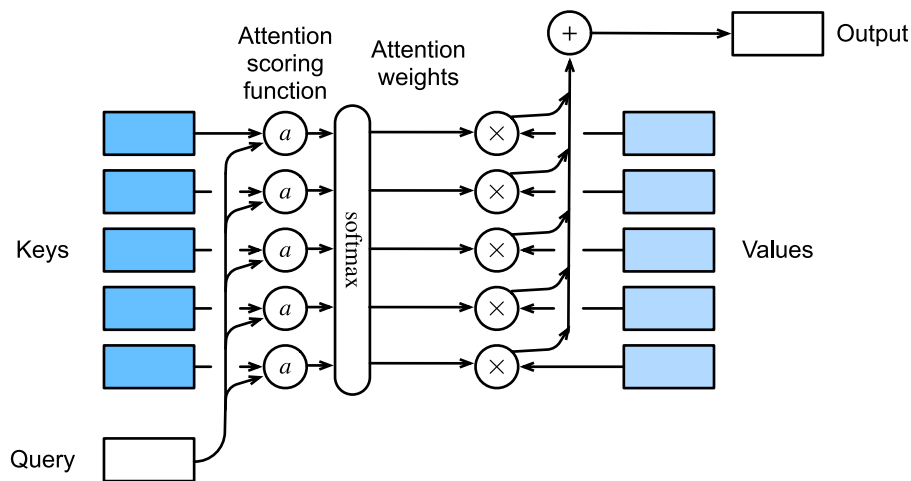
RNN seq2seq 模型与 attention 机制

但是我们怎么知道输出每一个 token 时应当关注哪一个 token 呢？没关系，因为在 AI 的世界里，一切都是可以学习或拟合的。

这里我们以目前最常用的 dot-product attention 为例，它的注意力计算公式就是：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

在其中 $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{n \times d_k}$, $V \in \mathbb{R}^{n \times d_v}$, 其中 n 是输入序列的长度, d_k, d_v 是两个超参数。 Q, K, V 三个矩阵都由输入的向量序列经过线性映射得到。



这里的 Q, K, V 分别代表 query, key, value 三个量，它们都来自数据库术语，人们输入 query 进行查询，通过与 key 进行比对后获取对应的 value。

Transformer 的时代

在约 2017 年时，人们开始在很多任务上探索一种自注意力 (self attention) 机制。如果将序列当中的每一个 token 都输入给原序列做 attention 计算的话，我们得到的输出就会是另一个相同长度的序列。在经过适当的训练之后，它就能完成很多我们期望的信息处理任务。

更进一步地，我们也可以并行地使用多套参数计算 self-attention，并将得到的结果叠加在一起，这就形成了多头注意力 (multi-head attention)。使用 self-attention 生成更好的序列表示之后，就可以更好地服务于 RNN 去做生成任务了。

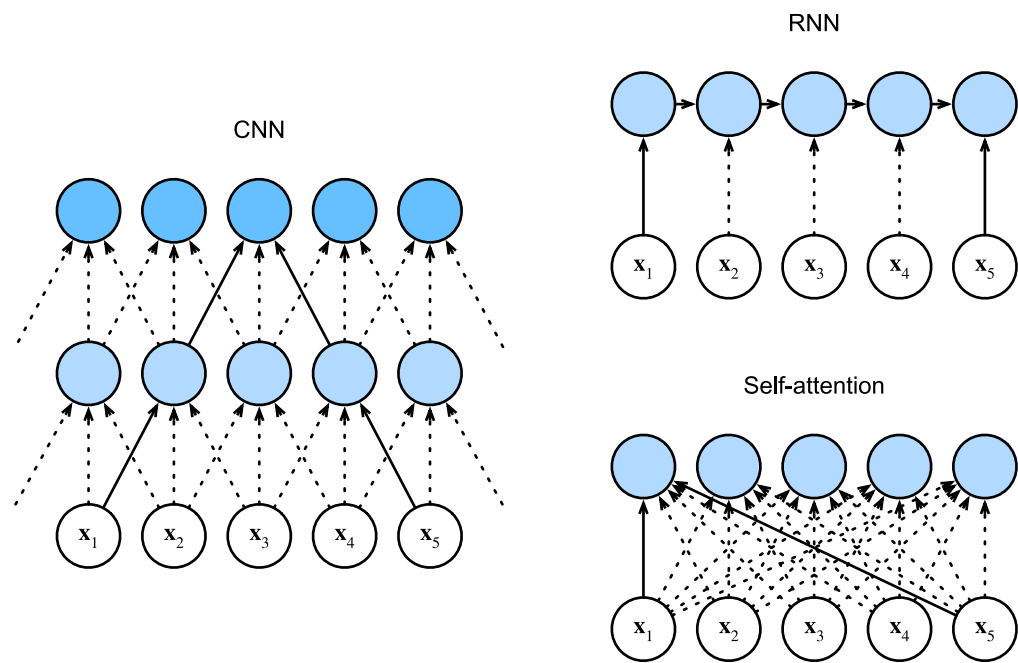
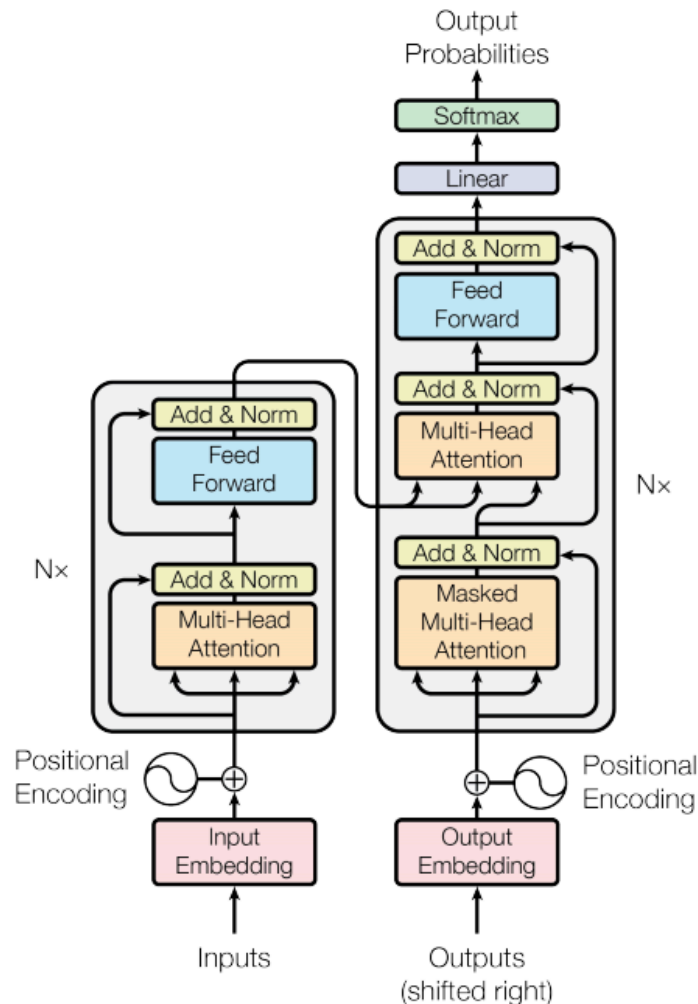


Figure 4: CNN, RNN 和 self-attention 三种序列计算模式的对比

既然单独使用 Attention 机制即可以处理序列数据，那么是否有可能抛弃 RNN 而只使用 Self-Attention 单飞呢？

于 2017 年发布的重量级论文 [Attention is All You Need](#) 就做出了这样的尝试，并且取得了相对 RNN 显著更优的表现，最终成为 AI 领域引用量第二高的论文。删去 RNN 之后纯 attention 堆叠得到的模型结构就是 Transformer，其优势具体体现在：

- 自然捕获全局依赖关系
- 能够大规模并行计算
- 实际展开深度显著更低，易于训练



最早的 Transformer 同样应用于机器翻译任务，并采用 Encoder-Decoder 架构实现。后续它的两个影响力最大的衍生分支，BERT 和 GPT 则分别仅使用了它的 Encoder 和 Decoder 部分。

Transformer 的 Encoder 和 Decoder 两块结构最主要的区别就在于 Attention 计算时是否具有因果掩码 (Causal Mask)。

因果掩码通过固定删除 attention 的上三角部分，确保了每一个 token 在计算时无法利用到后续序列的信息，如此才能正确执行自回归生成。

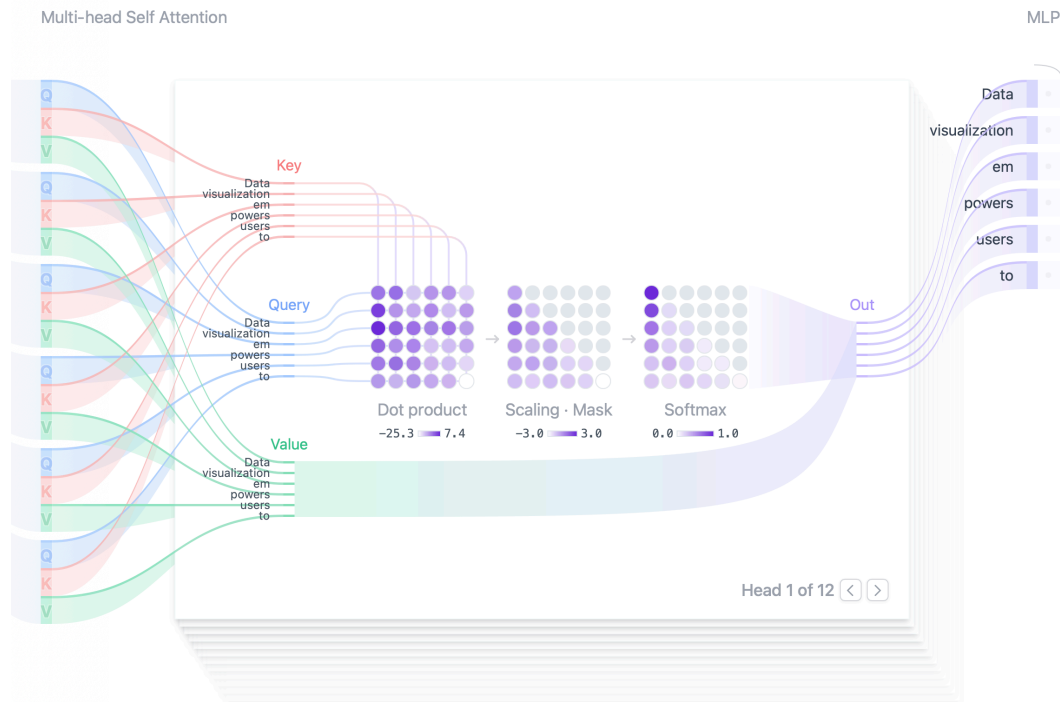


Figure 5: Transformer Decoder 部分计算示意图，参考自链接 [Transformer Explainer](#)，非常推荐阅读。

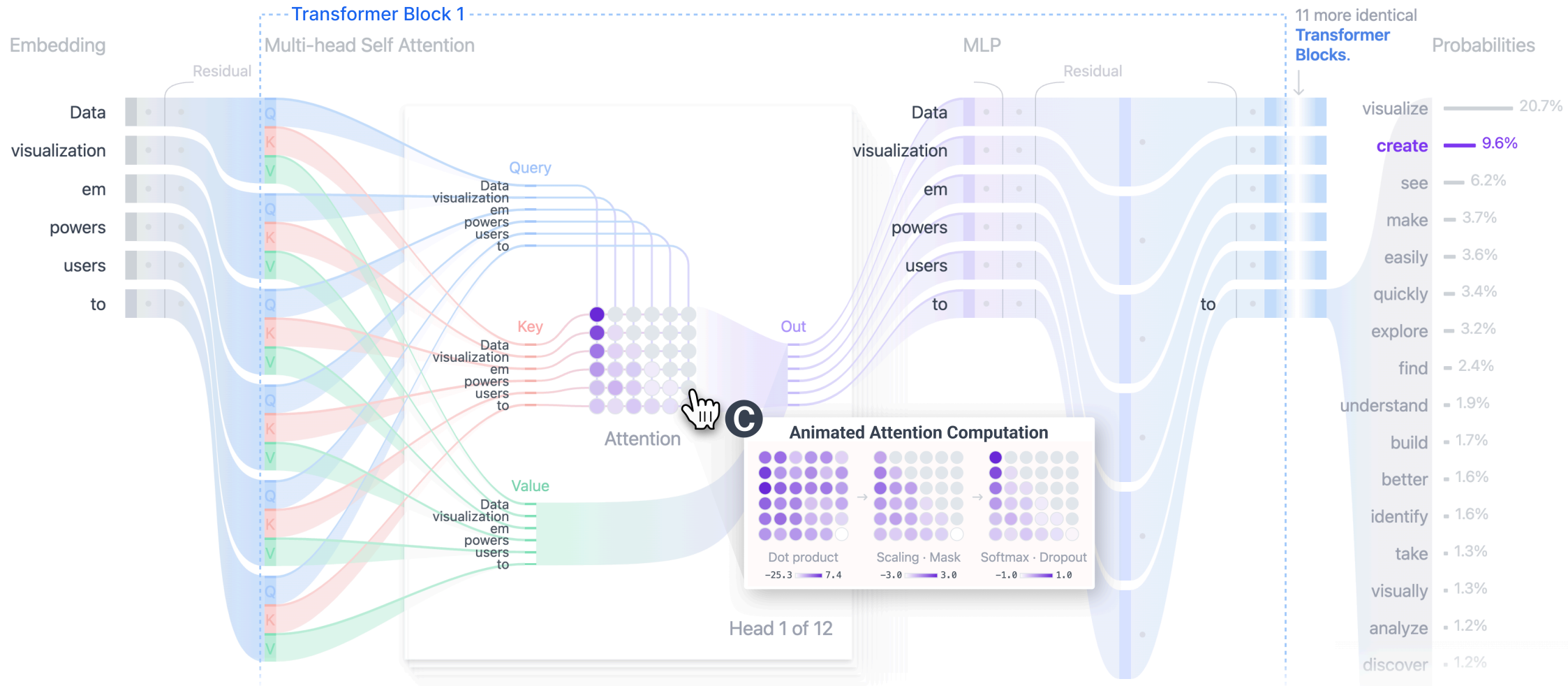
Examples ▾ Data visualization empowers users to **create** **A**

Data visualization empowers users to **create**

A

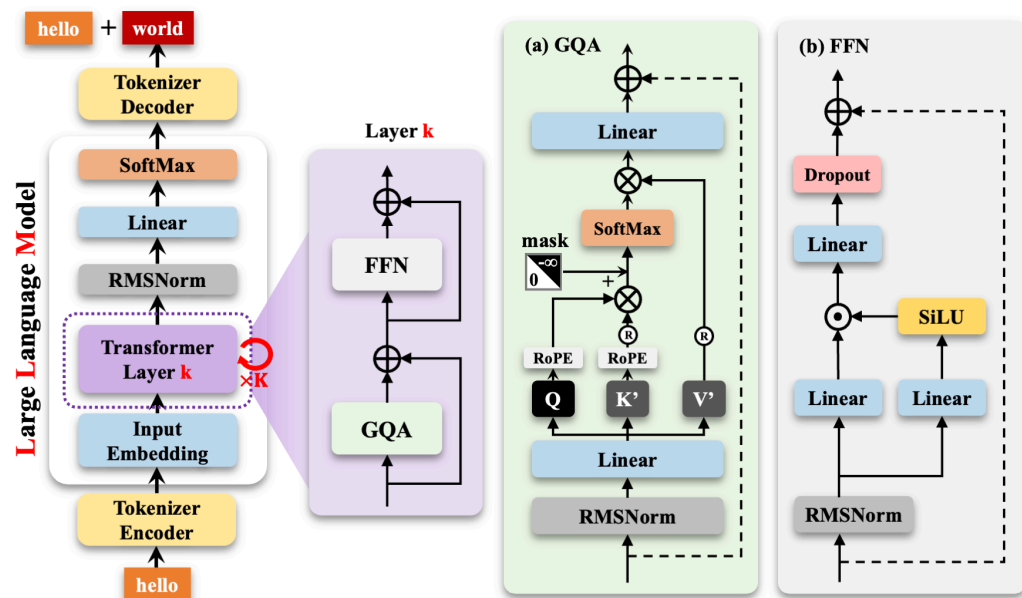
Generate

Temperature ?



我们也在代码示例“little_lm_demo.py”当中实现了一个具有三个 attention block 的 decoder-only 语言模型，并同样使用最小的字符级 tokenizer，并在莎士比亚全集上进行训练。最终得到的效果如下页 PPT 所示。

如果希望语言模型发挥更好的效果，我们仍需要使用更多的训练数据以及更完善的训练流程。



The Structure of MiniMind (Dense Model)

Figure 6: [MiniMind](#) 模型结构示意图。
MiniMind 是一个完整且现代化的小型 GPT 类语言模型的开源实现，如有兴趣非常推荐拿来玩一玩！

Epoch 0:

HUFwwnBDk wICnw'XrH UYxyo3RwRF.
PTVIzOPVL?sCg'Q Wf&NnwQBdh!:RF-
nN:sgoqedX,\$LIGsXLbBOw'oAgkk,ghurtEcf?
c:fYejfN\$wq.Qe-3dV-xn?'DcVg?GwC!TQ\$!
P&qM!F3bwwl?
A:bAl&qMjXsFlxY\$vxLbalOPSRtL3ryY!
BDleq:llawFFG?szhX

Epoch 20:

IUSINHA: Will fow frre couck.

VOLAUKE: Ry thou prom of ant us, ind wer
ca moorge?

ORUKEN ETHARD TINRY RENT: Whours:
Whe sply mus buresh, Rome your of
maaicest. Firch murthants andeow the our
ard, o

Epoch 50:

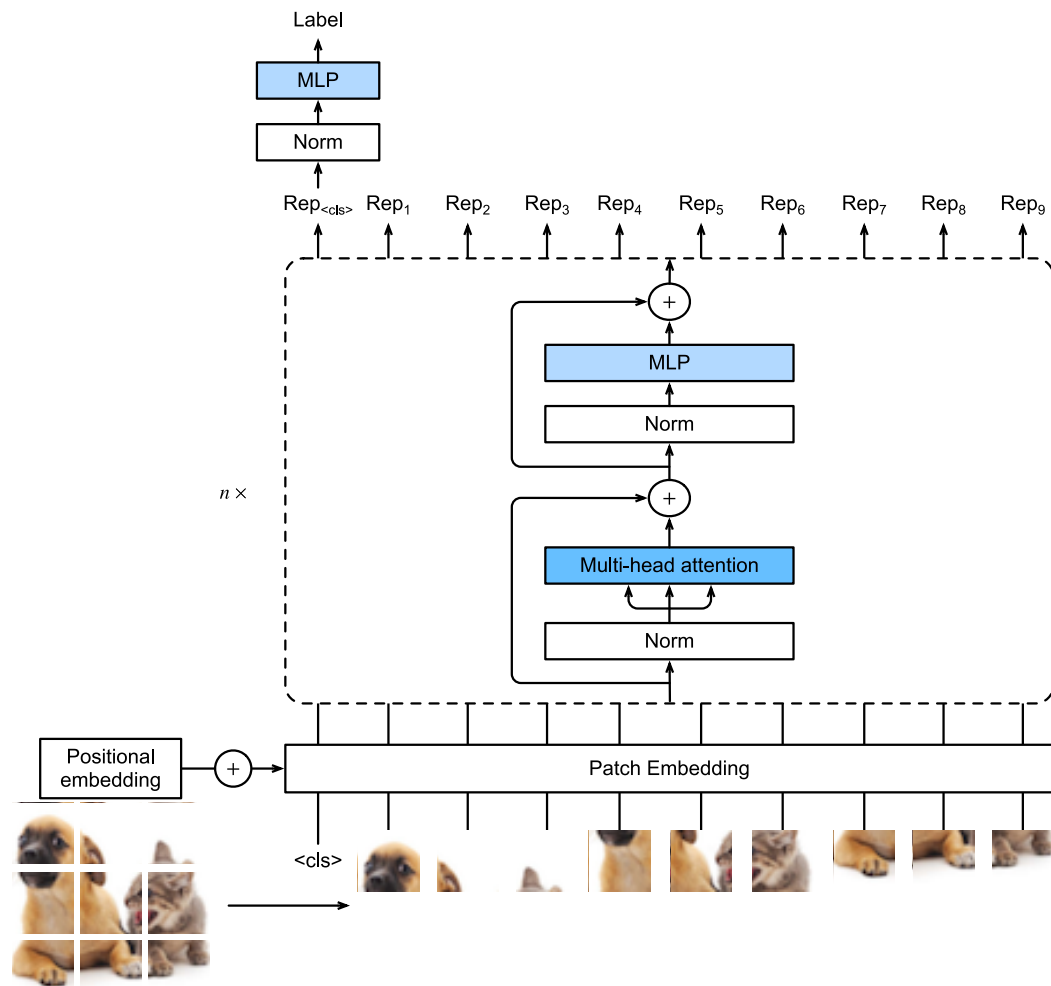
When thee I shalt, it prossived; in the the
roubled hold.

DUKE OF NEDYBEY: Nors and truck beauter
all our not a ofence, I'll the are way this
weath, when these now.

Cleto it the is becouranter, you

注意力机制在图像相关的任务当中同样具有重要的意义，因此当 Transformer 架构在 NLP 领域取得成功之后，将它迁移到 CV 领域就显得理所当然了。

于是在 2020 年，“*An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*”这篇论文当中就首次提出的 Vision Transformer (ViT) 架构，它首先将图像切分为数个固定大小的小 patch，此后再通过线性层或卷积层直接映射得到语义向量。之后就可以直接将图像当作 token 序列进行了。



由于在二维图像当中似乎并不存在自然的顺序关系，类似 GPT 的 Transformer 自回归生成范式就难以应用到图像生成领域。此前也存在使用扫描顺序逐块生成的工作，但是效果并不很好。

直到 2024 年年中时，由北京大学一位硕士生所做的 [*Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction*](#) (VAR) 首次提出了 Scale 顺序预测的概念，并应用 GPT-2 模型取得了极好的效果。

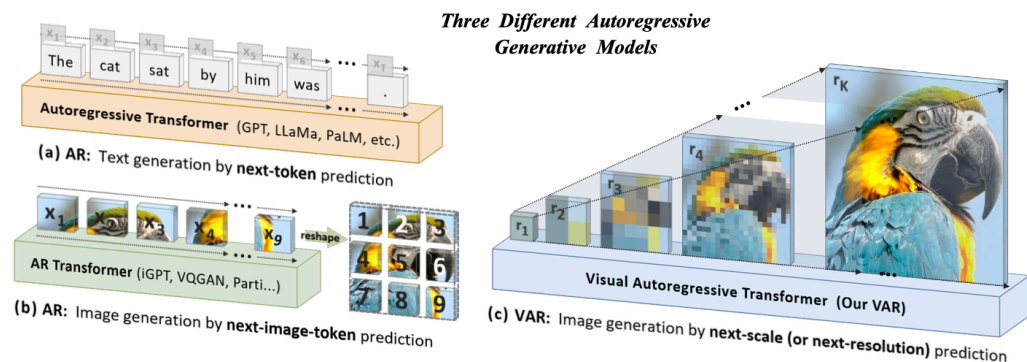


Figure 7: VAR 主页配图，其中也对比了彼时存在的三种 Transformer 生成框架的工作原理，其中 VAR 在生成效率和质量上均显著更高。

在 Transformer 架构取得显著成功的同时，其自身的缺点也逐渐凸显。当前 Transformer 架构明显的缺陷基本都在运行效率上：

- 在训练或推理的 prefill 阶段，Transformer 的计算复杂度与输入序列长度 n 之间呈平方关系 (每两个 token 之间需要计算一次相关性)
- 在推理生成阶段，使用 KV Cache 可以保留前期 token 计算的结果，将复杂度削减到线性的同时却显著提高了显存压力

这其中存在很多代表性的工程优化。

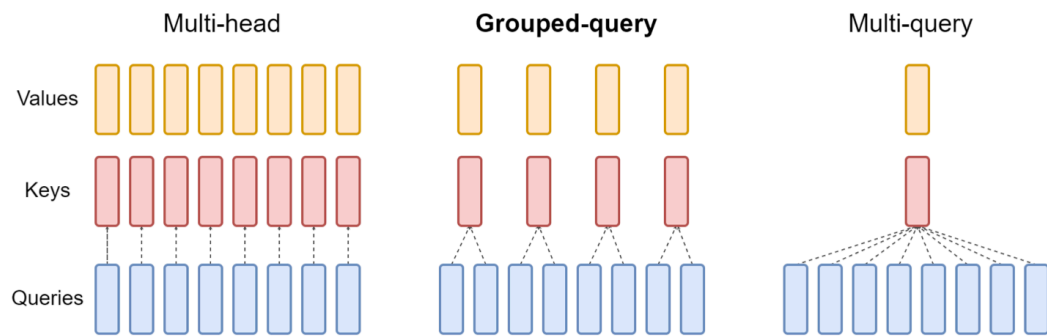


Figure 8: Grouped Query Attention (GQA) 技术，通过多头注意力共享 KV 值的方法显著削减了显存消耗，并保持性能较少下降。

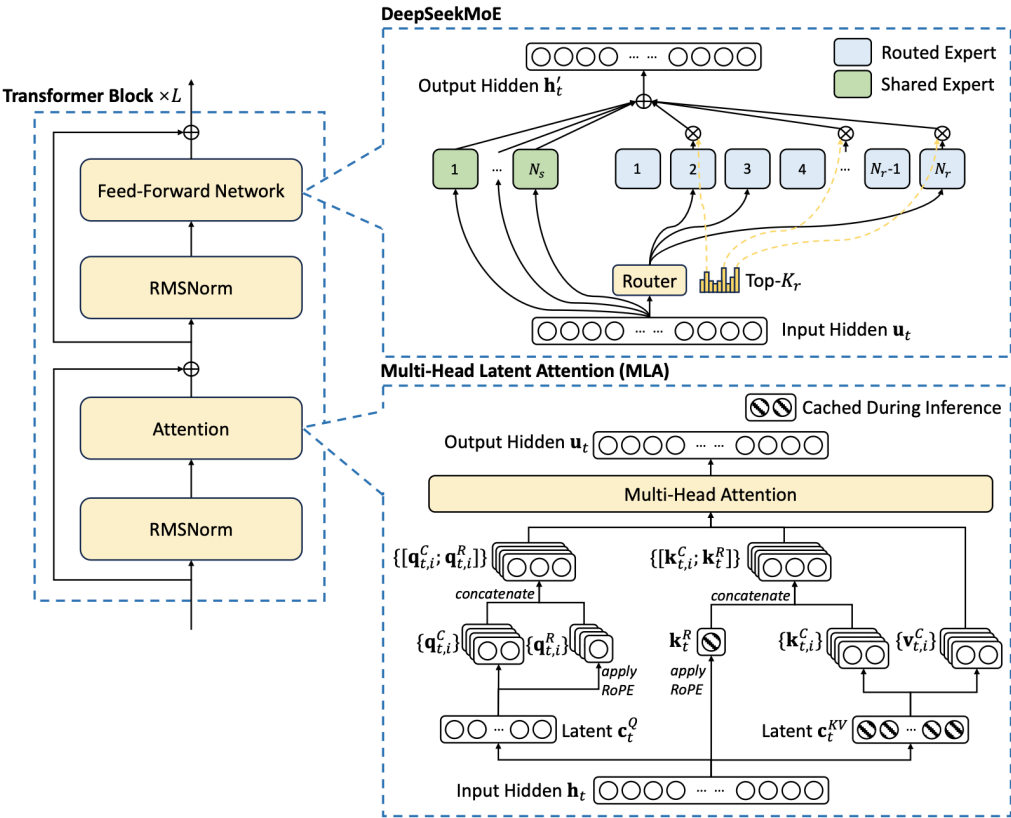


Figure 9: DeepSeek-V3 的模型架构，MLA, MoE 和 FP8 训练带来了成本的显著降低。

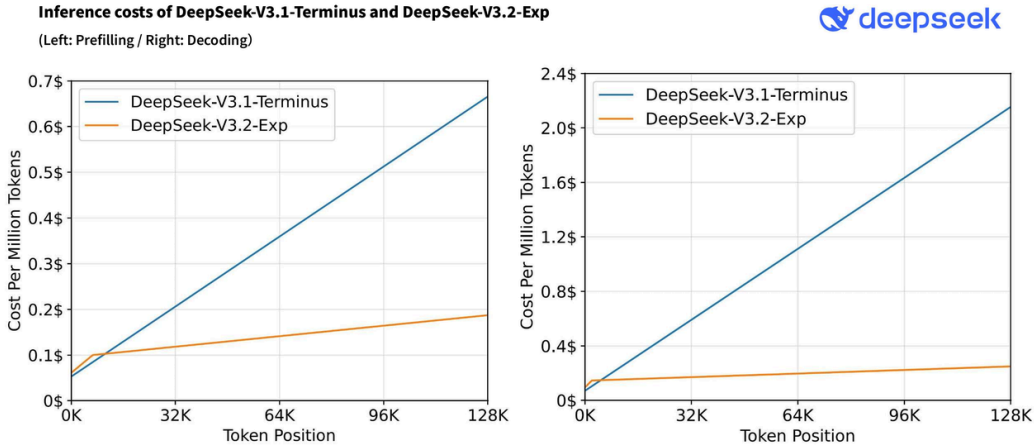


Figure 10: DSv3.2 采用 NSA 后成本大幅下降

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \text{Diag}(\boldsymbol{\alpha}_t) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top \in \mathbb{R}^{d_k \times d_v}; \quad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t \in \mathbb{R}^{d_v} \tag{1}$$

$$\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} = \left(\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} - \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \times \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \right) \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} + \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \times \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} = \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$$

Figure 11: Kimi KDA 线性注意力，类 RNN 层与 full attention 层混合同样有效

敬请期待下集 - 模型训练

接下来两次讨论班我们会主要讲解 AI 模型的训练流程，同时也会介绍模型结构当中的各种正则化层。大致介绍当今的 AI 模型完整的构建方法。